

Mixed-Criticality Systems, Resource Sharing, and Multiprocessor Systems

Pós-Graduação em Sistemas Computacionais Embebidos
Sistemas Operativos de Tempo-Real (PGSCE-SIOPTR)

2025/26

Paulo Baltarejo Sousa
`pbs@isep.ipp.pt`

Material and Slides

Some of the material/slides are adapted from various:

- Presentations found on the internet;
- Books;
- Web sites;
- ...

Gratitude

- Many thanks to Prof. Luís Miguel Pinho:
 - For providing course material
 - For helping to prepare this course
 - ...

Outline

1 Mixed-Criticality Systems

2 Resource sharing

3 Multiprocessor Systems

1 Mixed-Criticality Systems

2 Resource sharing

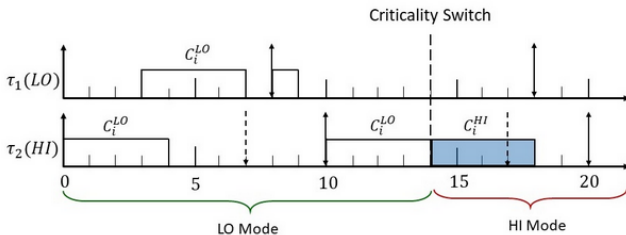
3 Multiprocessor Systems

Mixed-Criticality Systems (MCS) (I)

- A Mixed-Criticality System (MCS) is a real-time system that **comprises tasks having two or more distinct criticality levels** (e.g., safety-critical and mission-critical).
- Existing scheduling techniques reserve large amounts of computational resources to ensure that every real-time task, including those that are less critical, **provide full service even under theoretical worst-case conditions** (safety assurance).
- As a result, **the computational resources are highly under-utilized in such systems.**
- To overcome this limitation, **the mixed-criticality real-time task and scheduling model** was proposed, wherein all tasks are required to provide full service under normal (practical worst-case) conditions, but **less critical tasks may only be able to provide degraded service under theoretical worst-case conditions.**

MCS (II)

- The model was later extended to the notion of modes of execution (Adaptive Mixed Criticality)
 - During execution, the system uses the more “optimistic” best-case execution times of lower criticality levels
- If one the tasks overruns its best-case execution time, a mode change is triggered
 - Criticality level is increased
 - Tasks with level below the current level are dropped
 - Scheduling now considers the worst-case execution time values



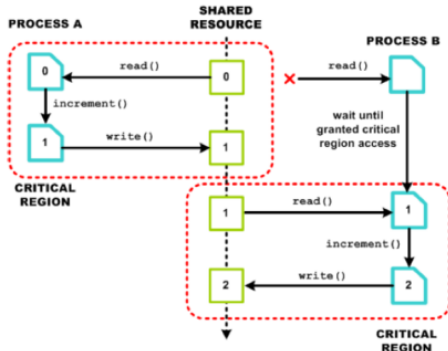
1 Mixed-Criticality Systems

2 **Resource sharing**

3 Multiprocessor Systems

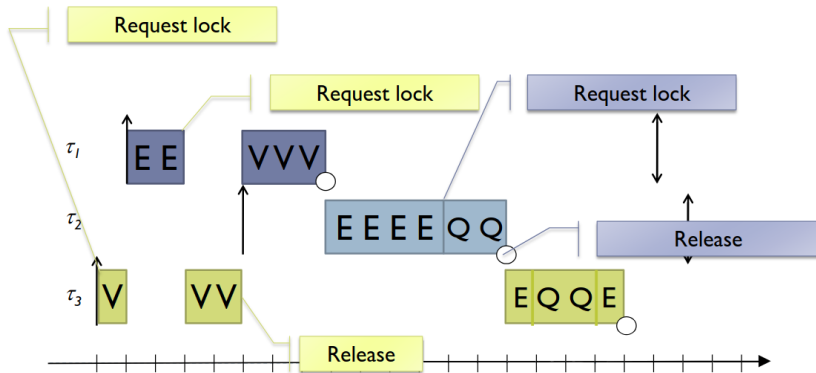
Lock

- In most systems, processes share resources apart from the CPU:
 - Files, memory areas, screen, printers, network and so on.
- Synchronisation mechanisms such as **locks are used to manage shared resources.**



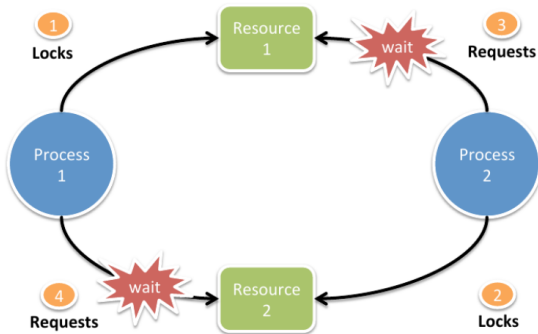
Example

	C (ms)	T=D (ms)	Priority	Arrival instant (ms)	Execution sequence
τ_1	5	17	3	1	EEVVV
τ_2	5	15	2	5	EEEEQQ
τ_3	7	35	1	0	VVVEQQE



Deadlock

- A condition involving one or more processes and one or more resources, such that each process waits for one of the resources, but all the resources are already held.
- Therefore, none of the processes can continue



Priority Inversion (I)

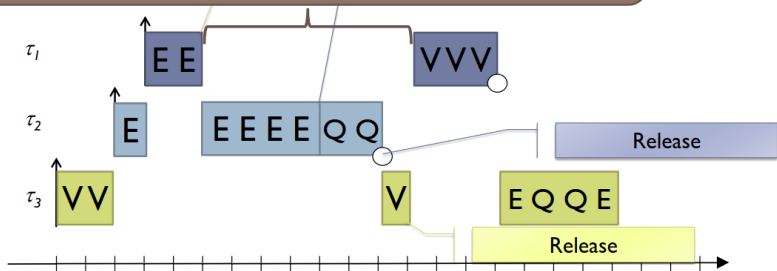
- In pre-emptive systems, in principle it should execute the task with the highest priority.
- However, **due to waiting in the access to locked resources, a high-priority task may need to wait for low-priority tasks having the lock on the resource** (and others of intermediate priority)
- This is a **priority inversion**

Priority Inversion (II)

	C (ms)	T=D (ms)	Priority	Arrival instant (ms)	Execution sequence
τ_1	5	17	3	3	EEVVV
τ_2	5	15	2	2	EEEEQQ
τ_3	7	35	1	0	VVVEQQE

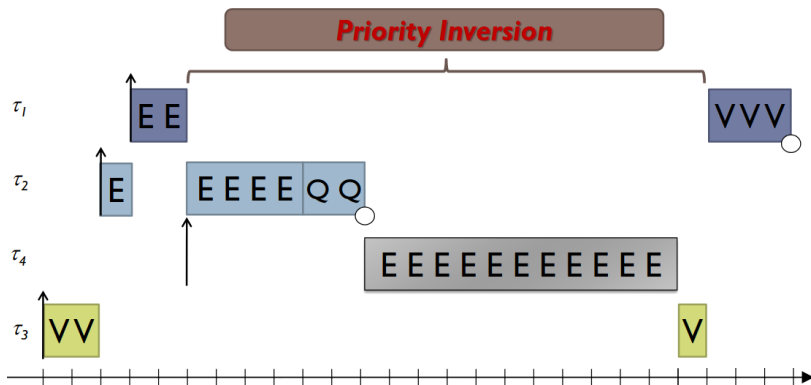
Priority Inversion

A higher priority task (τ_1) waits for a lower priority task (τ_3). And middle periodic tasks are allowed to execute.



Priority Inversion (III)

- If no rule is applied in sharing resources, it is **not possible to determine maximum blocking time** (or the blocking time is large)



Priority Inversion (IV)

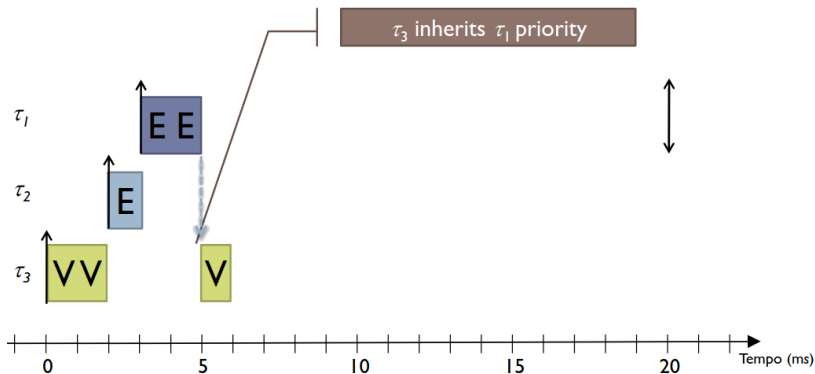
- Priority inversion **is inevitable in the presence of shared resources usage** (intrinsic to blocking)
 - However, we need to limit and **be able to quantify the impact in the worst-case response time** to guarantee schedulability of the task set
- To address this problem several protocols are used to manage shared resources
 - These protocols are used to grant exclusive access to a shared resource, but bounding the period of priority inversion, and being analyzable, i.e., allowing to determine the worst-case blocking that a task can have

Avoiding Priority Inversion

- Priority Inheritance Protocol (PIP)
 - Allows several blocking periods but guarantees maximum blocking period
- Original Ceiling Priority Protocol (OCP)
 - Allows only one blocking period but introduces unnecessary blocking
- Immediate Ceiling Priority Protocol (ICPP)
 - Also known as PCE - Priority Ceiling Emulation
 - Same as before, but simpler to implement
 - Also prevents deadlocks
- Stack Resource Policy (SRP)
 - Similar to IPCP but allows for dynamic priorities

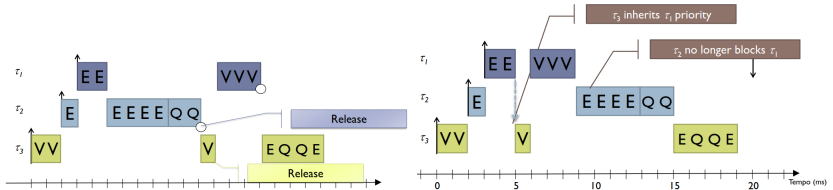
PIP (I)

- When a higher priority task is blocked in a shared resource, **the lower priority task executing in the resource "inherits" the higher priority while executing in the resource**



PIP (II)

	<i>C</i> (ms)	<i>T=D</i> (ms)	<i>Priority</i>	<i>Arrival instant (ms)</i>	<i>Execution sequence</i>
τ_1	5	17	3	3	EEVVV
τ_2	5	15	2	2	EEEEQQ
τ_3	7	35	1	0	VVVEQQE



PIP (III)

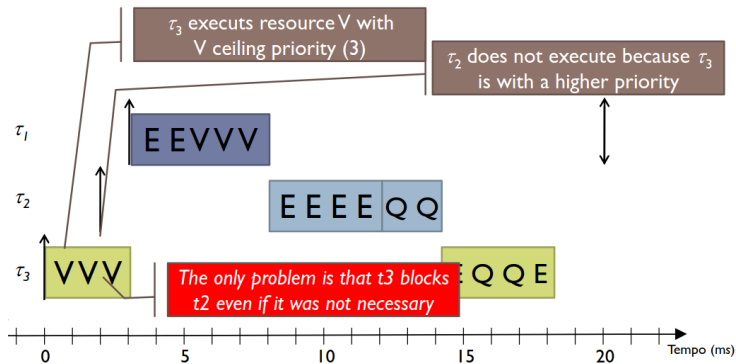
- With the protocol, **blocking still occurs when:**
 - A lower priority task **holds a resource the task needs** (direct blocking) or
 - A lower priority task **inherits a higher priority than the task because it holds a resource the higher-priority task needs** (push-through blocking)
 - But a higher priority task is only blocked once for each resource it uses
- Relatively easy to implement
- Suffers from chain blocking and does not prevent deadlocks

OCPP

- The OCPP is an extension of PIP
 - Additional rule about access to free resources, **inserted to guarantee that all required resources are free**
 - For each resource **a priority ceiling is defined, which equals the priority of the maximum priority task that uses it**
 - A task can only take a resource **if the resource is free and if the task priority is greater than the ceilings of all resources currently taken**
 - The rationale for this assignment rule lies in its ability to prevent chain blocking, where a low-priority task holding a resource indirectly delays multiple higher-priority tasks through preemption chains.
 - It only allows the access to the first resource when all other resources that a task needs are free
 - Note also that each task can only be blocked once
 - Due to its complexity, the immediate variant is usually preferred

IPCP (I)

- Resources are given a priority **which is equal to the highest priority of the tasks that can use the resource** (ceiling), which is determined before execution
- When executing in the resource tasks immediately execute with the priority of the resource (ceiling)

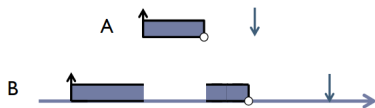


IPCP (I)

- It mitigates priority inversion by immediately elevating a task's priority to the ceiling of a resource upon locking it, irrespective of whether higher-priority tasks are pending.
- This proactive approach ensures that tasks holding resources cannot be preempted by medium-priority tasks during critical sections, providing bounded blocking essential for schedulability analysis.
- As a precursor to ICPP, the OCPP delays priority boosts until actual blocking occurs.

SRP

- SRP is considered a refinement of PCP, designed to work with both static (fixed) and dynamic priority scheduling
- Introduces a new concept, the **preemption level**
 - **The capacity of a task to cause preemption on another one** (static parameter)



For EDF, the preemption level can use the relative deadline as the static parameter

$$D_A < D_B \Rightarrow PL(A) > PL(B)$$

- The resource ceiling for a resource is now the highest preemption level of all tasks that may access the resource
- The system ceiling is the highest resource ceiling among all currently locked resources
- A task can preempt another task if it has the highest dynamic priority (earliest absolute deadline for EDF), and its preemption level is higher than the system ceiling

Linux

- Linux implements hierarchical scheduling, but with a different approach
 - Tasks with real-time scheduling class have precedence over non real-time (Fair Scheduler)
 - `SCHED_DEADLINE` (dynamic priorities) is always first
 - Fixed priority tasks share priorities 1 to 99
 - `SCHED_FIFO` (FIFO preemptive) and `SCHED_RR` (round robin)
- Linux does not implement EDF, but instead a server-based version (CBS like) with `SCHED_DEADLINE` policy
- However, for resource sharing, only priority inheritance is supported
- This means that resource sharing with `SCHED_DEADLINE` does not solve priority inversion, and schedulability analysis cannot be used

1 Mixed-Criticality Systems

2 Resource sharing

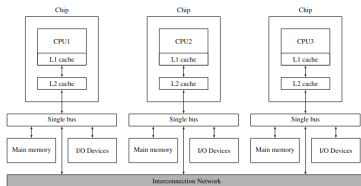
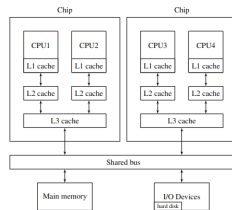
3 Multiprocessor Systems

Processors

- A multiprocessor can be classified according to the types of processors that compose it.
- A multiprocessor is said to be:
 - **Identical**, if all processors have the same features: clock speed, architecture, cache size and so on;
 - **Heterogeneous**, if they have different capabilities and different speeds.
 - In **uniform** multiprocessors, one processor may be x times faster than another one, with the meaning that it executes all possible tasks x times faster.

Memory

- According to the memory model, multiprocessor systems are classified as:
 - In a **shared memory model** all processors share the main memory.
 - In the **distributed memory model** each processor has its private memory and when remote data is required the processor has to communicate with respective remote processors, through an interconnection network.



Real Time Issues

As early as in the 1960s, it was observed by Liu, the inventor of RM and EDF, that

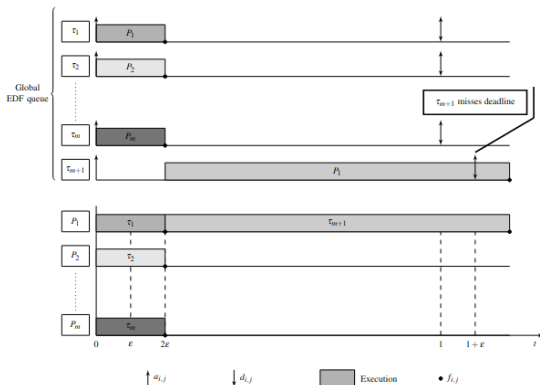
“Few of the results obtained for a single processor generalize directly to the multiple processor case; bringing in additional processors adds a new dimension to the scheduling problem. The simple fact that a task can use only one processor even when several processors are free at the same time adds a surprising amount of difficulty to the scheduling of multiple processors”.

Migration

- Multiprocessor systems introduce an additional dimension to the scheduling algorithm classification known as the **migration** degree.
- According to the migration degree, multiprocessor scheduling algorithms have traditionally been categorized as:
 - **Global scheduling algorithms** store all ready tasks (or jobs) in one global queue that is shared by all processors.
 - At any moment, the m highest-priority ready tasks among those are selected for execution on the m processors.
 - Tasks can migrate from one processor to another during its execution;
 - **Partitioned scheduling algorithms** divide the task set such that all tasks in a partition are assigned to the same processor thus, reducing a multiprocessor scheduling problem, at least conceptually, to a set of m uniprocessor problems.
 - Tasks are not allowed to migrate from one processor to another.
 - Partitioning implies assigning tasks to processors.

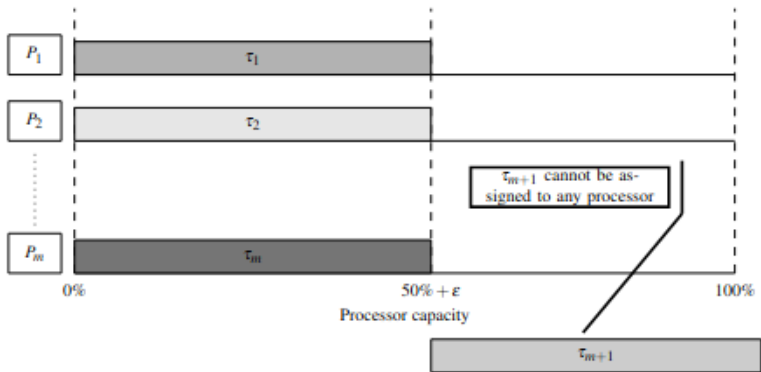
Dhall's effect

- It was found that RM and EDF were not fit for multiprocessors; deadlines can be missed although close to 0% of the computing capacity is requested
 - This is known as Dhall's effect, which occurs when a task set is composed by m low-utilization tasks and one high- utilization task.



Partition problem

- With $m \rightarrow \infty$ and assuming $\epsilon \rightarrow 0^+$, the least schedulable utilization of any partitioned scheduling algorithm decreases to 50%

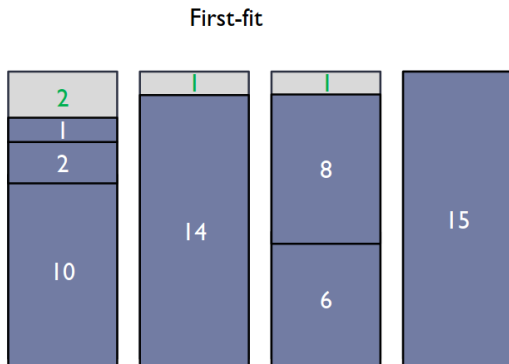


Bin packing (I)

- NP hard problem, so usually heuristics are used
 - **First-fit** (FF): The processors are considered ordered in some manner and the task is assigned to the first processor on which it fits
 - **Worst-fit** (WF): A given task is assigned to the processor with the maximum remaining capacity
 - **Best-fit** (BF): A given task is assigned to the processor with the minimum remaining capacity (but still has enough capacity to fit its weight)
 - Tasks can be ordered in an increasing or decreasing order, e.g.:
 - First-fit decreasing: Start with the tasks with higher utilization and assign to the first processor it fits
 - Best-fit increasing: Start with the tasks with lower utilization and assign to the processor with minimum remaining capacity
 - ...
 - And many others

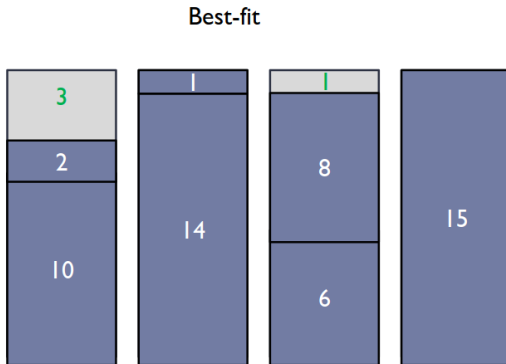
Bin packing (II)

- Heuristic: First-fit
- Bin capacity: 15
- Loads: 7
 - L1 = 10
 - L2 = 2
 - L3 = 14
 - L4 = 1
 - L5 = 6
 - L6 = 15
 - L7 = 8



Bin packing (III)

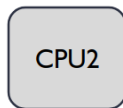
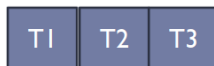
- Heuristic: Best-fit
- Bin capacity: 15
- Loads: 7
 - L1 = 10
 - L2 = 2
 - L3 = 14
 - L4 = 1
 - L5 = 6
 - L6 = 15
 - L7 = 8



Scheduling models (I)

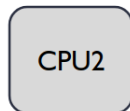
- **Global scheduling**

- Allows any job to execute on any given processor
- Job migrations are allowed



- **Partitioned Scheduling**

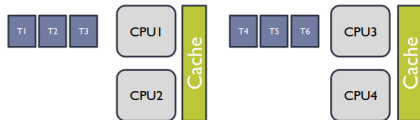
- A task is mapped to a particular processor and it is only allowed to execute its jobs on the processor to which it has been mapped
- Job migrations are not allowed



Scheduling models (II)

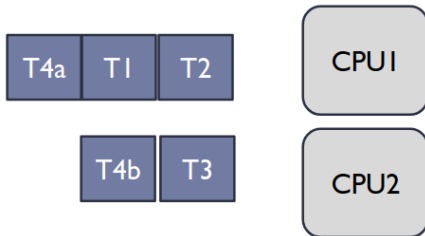
• Clustered Scheduling

- The processors of a given platform are partitioned in clusters and each task is mapped to one of those clusters
- Job migrations are allowed within a cluster



• Semi-partitioned

- A few tasks are allowed to migrate while others are mapped to processors (not allowed to migrate)
- E.g. task-splitting



Exercise

- Trace the execution of the following task set with 2 processors for 30 ms
 - Using Global EDF and Global RM
 - Using Partitioned EDF and Partitioned RM
 - Try First Fit or Best-fit decreasing

	Arrival (ms)	C (ms)	T=D (ms)
τ_1	2	5	10
τ_2	3	2	9
τ_3	2	1	5
τ_4	3	3	9
τ_5	0	7	12

Resource sharing

- In multiprocessor systems, data may be shared by tasks executing in different cores
- Necessary to have mechanisms to synchronize access to the shared data
 - Extensions of existing approaches, e.g.
 - Multiprocessor Priority Ceiling Protocol (MPCP)
 - Multiprocessor Stack Resource Policy (MSRP)
 - Or new protocols, e.g.
 - Multiprocessor resource sharing Protocol (MrsP)
- Goal is to maintain as much as possible **the properties of bounded priority inversion and free of deadlocks**